

Comparaison de 3 approches pour l'analyse de sentiments

Introduction

Cet article compare trois approches pour effectuer l'analyse de sentiments.

- L'utilisation d'une API azure retournant le sentiment d'un texte.
- L'élaboration d'un modèle simple avec azure machine learning studio.
- L'élaboration d'un modèle avancé avec réseaux de neurones.

I - API sur étagère

Présentation

Microsoft offre via sa plateforme azure un ensemble de services « Microsoft cognitive services » mettant à disposition des développeurs des services IA. Parmi ces services, une API permet la détection de sentiment.

Pour utiliser cette API, il faut posséder un abonnement azure et créer une ressource langage. Un point de terminaison et un clé sont associés à cette ressource pour pouvoir y accéder.

L'API est accessible via des bibliothèques clientes ou via l'API REST.

L'URL est composée du point de terminaison suivi du chemin du service à utiliser.

Le corps de la requête contient le texte à analyser et un id par texte au format JSON.

Il est possible d'envoyer dix textes à la fois.

L'API renvoie l'id, le sentiment et des scores

Ces scores sont un pourcentage réparti entre positif, négatif, neutre et mixed.

Le sentiment contient celui du score le plus élevé.

Test

Dans notre jeu de données, nous n'avons que des sentiments positifs ou négatifs.

Pour les sentiments neutre, celui ayant le score le plus élevé entre positif et négatif a été sélectionné.

Des tests avec 1600 tweets ont été effectués avec/sans preprocessing et avec/sans stemmarisation/lemmatisation.

Résultats

	preprocess_type	accuracy	azure_sentiment_time
0	without_clean	0.742500	57.320783
1	clean_only	0.719045	61.546694
2	stem	0.675676	61.563812
3	lem	0.702074	60.604272

Nous obtenons un score d'environ 0,7 pour un temps de réponse d'environ 1 minute.

Le preprocessing n'améliore pas les résultats.

II - Modèle simple avec Azure Machine Learning Studio

Présentation

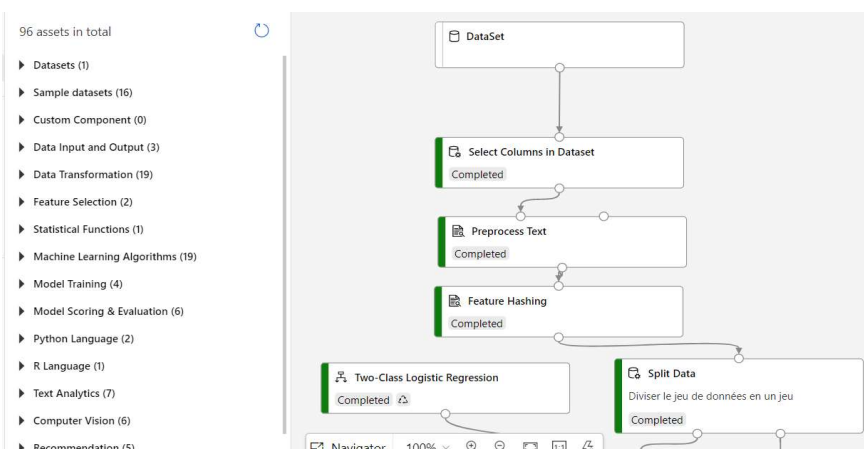
Azure machine learning studio permet de créer un pipeline complet des données brutes à l'évaluation d'un modèle, le tout graphiquement, en « drag'ndrop », sans ligne de code.

Il faut avoir créé auparavant un espace de travail, une cible de calcul et un dataset.

Construction du pipeline

Nous allons ici créer un modèle simple de classification supervisée, appliqué au traitement du langage.

Préparation des données



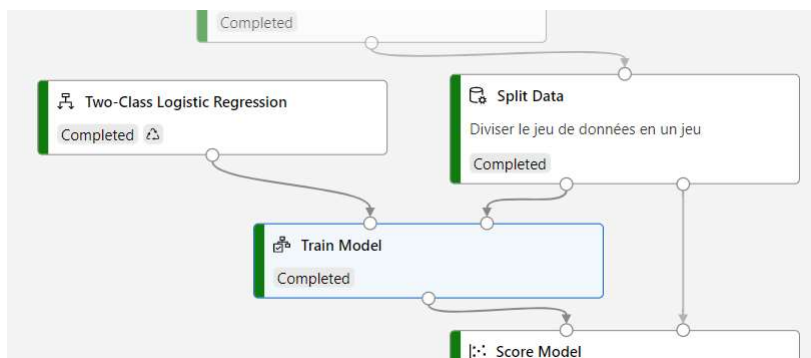
Il suffit d'effectuer un drag'n drop du composant sur la gauche pour l'intégrer au pipeline sur la droite.

Sélectionner le dataset préalablement enregistré et le composant de sélection des colonnes et sélectionner le texte et le label.

Sélectionner dans le composant de preprocessing du texte les fonctions à appliquer (contractions, stop-words, lemmatisation, caractères spéciaux...).

Sélection du composant « feature hashing » pour la vectorisation.

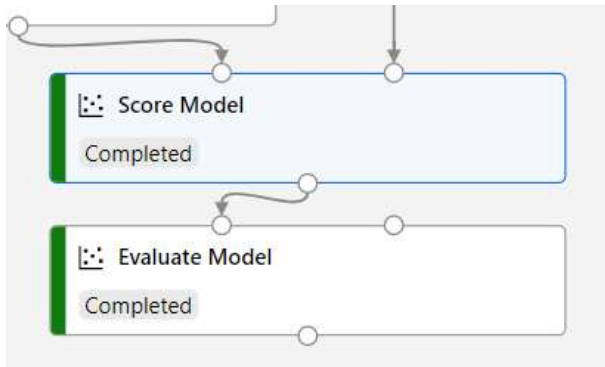
Modélisation



Choisir le modèle, ici une régression logistique, pour la classification binaire et split data pour la division du jeu de données en un jeu d'entraînement et de test

Ensuite, dans le composant pour entraîner le modèle, relié en entrée avec les deux composants précédents et configurer le label comme cible.

Évaluation



Relier les entrées de Score model au modèle et à aux données test de split data puis la sortie à Evaluate model.

Exécution du pipeline

Cliquer sur envoyer, configurer l'exécution du pipeline. Créer une expérience puis sur envoyer

Résultats

Properties	
Status	✓ Completed
Created	Jan 28, 2022 2:38 PM
Started	Jan 28, 2022 2:38 PM
Duration	2m 29.84s
Compute duration	2m 29.84s

Tags	
① No tags	

Metrics	
AUC	F1 Score
0.592	0.553
Accuracy	Lift curve
0.537	Type: Table
Confusion matrix	View all metrics
Type: Table	

Avec un dataset de 1000 tweets, l'exécution a duré environ 2,5 minutes pour un score de 0,53.

III – Modèle avancé

1 - Test de deux réseaux de neurones

Un réseau basique avec couches denses et un réseau LSTM sont testés.

Les LSTM, intégrant une fonction mémoire, sont adaptés à l'analyse des textes.

Les réseaux sont construits avec l'objet « Sequential » permettant d'empiler des couches séquentiellement.

La première couche du réseau est une couche « embedding » permettant d'intégrer un plongement de mots

Les plongements de mots permettent une représentation vectorielle des mots en prenant en compte leur contexte, leurs similarités.

Les couches suivantes sont les couches « dense » pour le modèle basic et « LSTM » pour le deuxième.

La couche de sortie est une couche à un neurone avec activation sigmoid pour la classification binaire

```
from keras.models import Sequential
from keras.layers import Dense
from keras.layers.embeddings import Embedding
from keras.layers import Flatten
from keras.layers import Activation, Dense, Dropout, Embedding, Flatten, Conv1D, MaxPooling1D, LSTM

def basic_nn_model(embedding):

    model = Sequential()
    model.add(embedding)
    model.add(Flatten())
    model.add(Dense(16, activation='relu'))
    model.add(Dense(16, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
    model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

    return model

def lstm_model(embedding, dropout=0.2, recurrent_dropout=0.2):

    model = Sequential()
    model.add(embedding)
    model.add(LSTM(100, dropout=dropout, recurrent_dropout=dropout))
    model.add(Dense(1, activation='sigmoid'))
    model.compile(loss = 'binary_crossentropy', optimizer='adam', metrics = ['accuracy'])

    return model
```

Les données en entrée doivent être préparées pour la couche embedding. Après preprocessing, les données texte sont transformées en séquences de tokens de taille fixe avec la librairie Tokenizer.

```
#Tokenize the sentences
tokenizer = Tokenizer()

#preparing vocabulary
tokenizer.fit_on_texts(list(x_train))

#converting text into integer sequences
x_train = tokenizer.texts_to_sequences(x_train)
x_test = tokenizer.texts_to_sequences(x_test)

#padding to prepare sequences of same length
x_train = pad_sequences(x_train, maxlen=seq_length)
x_test = pad_sequences(x_test, maxlen=seq_length)
```

Ensuite, la couche embedding est créée avec la taille du vocabulaire généré par le tokenizer, la taille des données en entrée et la taille du vecteur de sortie de l'embedding.

```
vocab_size = len(tokenizer.word_index) + 1
embedding_vector_length = 32
input_length = 300
embedding = Embedding(
    vocab_size,
    embedding_vector_length,
    input_length = input_length)
```

Le modèle est ensuite compilé, entraîné et évalué.

```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
hist_basic = model_basic.fit(x_train, y_train, validation_split=0.1, epochs=20, batch_size=128, verbose = 0)
scores = model_basic.evaluate(x_test, y_test, verbose=0)
```

Résultat :

Le modèle LSTM est plus long à entraîner et a obtenu une meilleure accuracy que le modèle basic.

2 – Test LSTM avec stemmatisation et lemmatisation

La stemmatisation garde la racine du mot, la lemmatisation est plus informative, prend le contexte...
Le modèle LSTM est testé avec des données en entrée traitées avec ces fonctions.

Ces traitements n'ont pas apporté d'amélioration significative.

3 – Test LSTM avec des plongements de mots pré-entraînés.

Les plongements de mots pre-entraînés sont entraînés sur de très grandes quantités de données.
Deux plongements de mots sont testés : Word2Vec (Google) et GloVe (Stanford University)

Il suffit d'associer les mots de notre dictionnaire aux vecteurs de mots fournis correspondant.

```
vocab_size = len(tokenizer.word_index) + 1
embedding_matrix_w2vec = np.zeros((vocab_size, 300))

for word, i in tokenizer.word_index.items():
    if word in w.index_to_key:
        embedding_matrix_w2vec[i] = w.get_vector(word)
```

```
embedding_vector_length = 300
input_length = 300
embedding_w2v = Embedding(
    vocab_size,
    embedding_vector_length,
    weights=[embedding_matrix_w2vec],
    input_length=input_length,
    trainable=False)
```

Résultat :

Le modèle est plus rapide à entraîner que sans plongement pré-entraîné, notamment avec GloVe dont la version avec une dimension de 100 a été testée (contre 300).

L'accuracy est aussi meilleure avec GloVe.

4 - Test de BERT

BERT est un réseau de neurones pré-entraîné.

L'étape de fine-tuning consiste en ré-entraînant le modèle avec nos données et notre besoin (classification binaire) afin d'adapter les poids.

BERT possède ses propres librairies BertTokenizer...permettant d'effectuer les étapes décrites précédemment.

Pour une classification binaire, BertForSequenceClassification est utilisé.

L'entraînement du modèle est long car il y a beaucoup de paramètres.

L'accuracy est plus faible qu'avec le modèle LSTM.

Il faudrait entraîner le modèle sur beaucoup plus de données.

Conclusion

	Temps de Développement	Temps D'entraînement	Durée Prédictions	Tuning	Performances
API sur étagère	++	+	-	-	-
Modèle simple	+	+	+	+	-
Modèle avancé	-	-	+	++	++

L'API permet d'obtenir de bons résultats et ne nécessite pas de développement.

Le modèle simple avec drag'n drop permet facilement et rapidement de développer un modèle.

Le modèle avancé offre de nombreuses possibilités de tuning afin de permettre d'obtenir de meilleures performances.